哈爾濱工程大學

第 05 讲 / 全 17 讲

# C 语言 for 循环语句用法详解

让计算机替你把重复的事做上一万遍

**三要素**

初始值 · 条件 · 步进

**执行流程**

初始化只做一次，然后转圈

**累加累乘**

先置 0 / 先置 1 的套路

**嵌套循环**

外层走一步，内层走一圈



主讲教师：郎大鹏

# 本节目标

哈爾濱工程大學

1. 说出 for 的三要素：初始值、条件、步进
2. 讲清执行流程：初始化只做一次，之后判断→执行→步进
3. 看懂计数变量 i，会算循环次数：((终值−初值)/步长 +1)
4. 掌握累加器（先置 0）与累乘器（先置 1）的固定套路
5. 会写嵌套 for：外层走一步，内层走一圈
6. 把 scanf 放进循环，处理一批输入数据

# 01

# 基本语法

为什么需要循环 · for 三要素 · 执行流程

计数变量 · 循环次数公式 · 用 printf 观察过程

哈爾濱工程大学

# 为什么需要循环？



程序里经常要重复做同一件事。重复 5 次还能手抄，重复 100 次、 10000 次就只能交给循环。

```
// 笨办法：写 5 遍
printf("第 1 遍：你好");
printf("第 2 遍：你好");
printf("第 3 遍：你好");
printf("第 4 遍：你好");
printf("第 5 遍：你好");
// 改成 100 遍要抄 100 行
// 改一遍内容要动 100 处
```

```
// 巧办法：交给 for
int i;
for (i = 1; i <= 5; i++) {
    printf("第 %d 遍：你好", i);
}
// 改成 100 遍只改一个数字
// i <= 100 就完事
// 内容只写一次，永不重复
```

**结论：凡是「同一件事重复做、只有某个数在变」的场景，就该用循环。**

# for 的三要素

括号里三段，用两个分号隔开

哈尔滨工程大学

```
for (i = 1 ; i <= 5 ; i++)
//   初始值      条件      步进
    printf("第 %d 遍: 你好", i);
//   上面这行是循环体，重复执行的部分
```

第 1 段

## 初始值 i = 1

`i = 1`

整个循环开始前只做一次，给计数变量一个起点。

起点不同，循环次数就不同。

第 2 段

## 条件 i <= 5

`i <= 5`

每轮循环之前都要判断一次。

成立才进循环体，不成立就结束循环。

第 3 段

## 步进 i++

`i++`

循环体执行完之后才做，把 i 往前推一步。

没有步进，条件永远成立，就停不下来。

# for 的执行流程

核心原理

哈爾濱工程大學

```
for (i = 1; i <= 5; i++)
    printf(" 你好 ");
// 逐轮追踪：
i=1  1<=5  成立  打印  i→2
i=2  2<=5  成立  打印  i→3
i=3  3<=5  成立  打印  i→4
i=4  4<=5  成立  打印  i→5
i=5  5<=5  成立  打印  i→6
i=6  6<=5  不成立 → 结束
// 循环体一共执行 5 次
// 结束时 i 的值是 6
```

## 四步转圈图

① 初始化 i = 1（只做一次）

↓

② 判断条件 i <= 5？

↓

③ 成立 → 执行循环体

printf(" 你好 ");

↓

④ 步进 i++

🔁 回到 ② 继续判断，不成立才结束

# 计数变量 i 的职责

i 是循环的「计数器」

```
// i 的三件事：起点、判据、被谁改
int i;
for (i = 0; i < 5; i++) {
    printf("%d ", i);
}
printf("结束 i = %d", i);
// 屏幕输出：
0 1 2 3 4 结束 i = 5
// i 依次取 0 1 2 3 4，共 5 个值
// 循环结束时 i 被多加了一次，是 5
```

## ① 起点由你定

i 在第 1 段拿到初值。从 0 开始还是从 1 开始，决定了后面所有写法。

## ② 只有步进能改 i

循环体里不要再写 i = i + 1 之类的改动，交给第 3 段统一处理。

## ③ 结束后 i 多走一步

退出时 i 的值刚好是让条件第一次不成立的那个数，不是最后一个有效值。

# 循环次数怎么算

通用公式

**次数 = ( ( 终值 − 初值 ) / 步长 ) + 1**

终值 = 条件成立时 i 能取到的最后一个值

```
for (i = 1; i <= 10; i++)
```

**10 次** 终值 10 ，初值 1 ，步长 1 → (10−1)/1+1 = 10

```
for (i = 0; i < 10; i++)
```

**10 次** 写 < 10 时终值是 9 → (9−0)/1+1 = 10

```
for (i = 2; i <= 10; i += 2)
```

**5 次** 步长变成 2 → (10−2)/2+1 = 5

```
for (i = 10; i >= 1; i--)
```

**10 次** 倒着走，步长算 1 → (10−1)/1+1 = 10

# 用 printf 观察循环过程

看得见才学得会

哈尔滨工程大学

```
// 把 i 和它的平方一起打印出来
int i;
for (i = 1; i <= 5; i++) {
    printf("i=%d  i*i=%d", i, i*i);
}

// printf 写在循环体里面
// 循环转几圈，它就打印几次

// 想看 i 走到哪一步
// 就把 i 放进 %d 的位置
```

## 运行结果

```
i=1  i*i=1
i=2  i*i=4
i=3  i*i=9
i=4  i*i=16
i=5  i*i=25
```

## 调试心法

循环是「看不见」的：它一瞬间就跑完了。

在循环体里加一句 printf 把关键变量打出来，执行过程立刻一目了然。

结果不对时，先打印，再改代码。

# 02

# 深入理解

累加器 · 累乘器 · 变步长

循环里嵌 if · 嵌套 for · 星号三角形

哈爾濱工程大学

# 累加器：先置 0，循环里 +=



```
// 计算 1 + 2 + ... + 100
int i, sum;
sum = 0;
// ↑ ① 先置 0，在循环外面
for (i = 1; i <= 100; i++) {
    sum = sum + i;
// ↑ ② 循环里累加
}
printf("sum = %d", sum);
// 输出 sum = 5050
// sum += i 就是 sum = sum + i
```

## ① 先置 0

sum = 0; 必须写在 for 之前。

写进循环体里，每轮都被清零。

## ② 循环里 +=

sum = sum + i; 加这一轮的 i。

简写为 sum += i;

## ③ 用结果在循环外

循环结束后 sum 才是最终答案。

printf 写循环里会打 100 行。

# 累乘器：先置 1

和累加器是一对双胞胎

```
// 计算 5! = 5*4*3*2*1
int i;
double fact;
fact = 1;
// ↑ ① 先置 1，不是 0
for (i = 1; i <= 5; i++) {
    fact = fact * i;
// ↑ ② 循环里累乘
}
printf("5! = %.0f", fact);
// 输出 5! = 120
```

## ① 起点必须是 1

乘法里 1 表示「还没乘任何数」。

要是写成 fact = 0 ，结果永远是 0 。

## ② 循环里 *=

fact = fact * i; 把这一轮的 i 乘上去。

简写为 fact *= i;

## ③ 用 double 装大数

阶乘涨得极快， int 装到 13! 就溢出变负数。

所以累乘器一般声明成 double 。

# 变步长：i += 2

第 3 段不一定是 i++

```
// 求 1~100 中所有偶数的和
int i, sum;
sum = 0;
for (i = 2; i <= 100; i += 2) {
// 起点 2，步长 2
    sum = sum + i;
}
printf("偶数和 = %d", sum);
// 输出 偶数和 = 2550
// i 的取值：2 4 6 8 ... 100
// 一共 (100-2)/2+1 = 50 次
```

## ① 步长写在第 3 段

i++ 只是 i += 1 的简写。

改 i += 2 就隔一个取一个。

## ② 起点决定落在谁身上

从 2 起步、步长 2 → 全偶数。

换成从 1 起步就全奇数。

## ③ 循环次数跟着变

步长翻倍，次数减半：100 次变 50 次。

套公式 ((终值−初值)/步长 +1)。

# 循环里嵌 if ：边走边挑



**任务** 找出 1~20 中所有能被 3 整除的数并打印出来

```
int i;
for (i = 1; i <= 20; i++) {
    //  每一轮都先问一句
    if (i % 3 == 0) {
        printf("%d ", i);
    }
}
//  输出  3 6 9 12 15 18
// i % 3  是  i  除以  3  的余数
//  余数为  0  就是能整除
```

## 运行结果

```
3 6 9 12 15 18
```

## 分工很清楚

for 负责「一个一个走过去」，把数送到面前。

if 负责挑出符合条件的，不满足就跳过。

## 循环次数不变

循环照样跑满 20 次，有 14 轮不打印。

# 嵌套 for ：外层走一步，内层走一圈

```
//  九九乘法表的前奏：打一遍坐标
int i, j;
for (i = 1; i <= 3; i++) {
    //  外层：控制行
    for (j = 1; j <= 4; j++) {
        //  内层：控制列
        printf("%d-%d ", i, j);
    }
}
// 1-1 1-2 1-3 1-4
// 2-1 2-2 2-3 2-4
// 3-1 3-2 3-3 3-4
```

## ① 走法口诀

外层 i 挪一步，内层 j 跑一整圈。

i 共挪 3 次，j 每次跑 4 个。

## ② 总次数 = 外层 × 内层

3 × 4 = 12 次，不是 3 + 4 = 7 次。

嵌套一层就乘一次。

## ③ 变量不能同名

外层用 i，内层用 j。

两层都用 i 就会互相干扰。

# 星号三角形

嵌套循环第一个看得见的成果

哈尔滨工程大学

**任务** 打印 5 行星号：第 1 行 1 颗，第 2 行 2 颗 …… 第 5 行 5 颗

```
// 第 i 行打印 i 颗星
int i, j;
for (i = 1; i <= 5; i++) {
    // 外层：一共 5 行
    for (j = 1; j <= i; j++) {
        // 内层：这行有几颗
        printf("*");
    }
    printf("%c", 10);
    // 10 是换行符的 ASCII 码
}
```

## 运行结果

```
*
**
***
****
*****
```

## 关键：内层终点跟着外层变

内层写成 j <= i ，而不是写死的 j <= 5 。

i 这一轮是多少，内层就转多少圈，这一行就有多少颗星。

想倒三角，把外层改成 i-- 即可。

# 03

# 循环与输入结合

读 n 个数累加 · 擂台模型找最大 · 求平均

乘法口诀表 · 水仙花数 · 鸡兔同笼

哈爾濱工程大學

# 读入 n 个数再累加

scanf 走进循环体

哈尔滨工程大学

```
// 先读个数 n，再读 n 个数求和
int n, i, x, sum;
scanf("%d", &n);
// ↑ 循环外先读个数
sum = 0;
for (i = 1; i <= n; i++) {
    scanf("%d", &x);
    // 每轮读一个新数
    sum = sum + x;
}
printf("总和 = %d", sum);
```

## ① 个数在外面读

scanf 读 n，决定循环转几圈。

n 是多少，就转多少圈。

## ② 数据在里面读

scanf 放进循环体，每轮读一个。

读一个、加一个，读完刚好加完。

## ③ 别忘了 &

scanf 里变量名前要加 &。

写成 scanf("%d", x) 是错的。

# 擂台模型：找最大值

第一个数先当擂主

哈爾濱工程大學

**任务** 读入 n 个整数，找出其中最大的那个并输出

```
int n, i, x, max;
scanf("%d", &n);
scanf("%d", &max);
// ↑ 第一个数先当擂主
for (i = 2; i <= n; i++) {
    scanf("%d", &x);
    if (x > max) {
        max = x;
        // 挑战成功，换擂主
    }
}
printf("最大 = %d", max);
```

### ① 擂主先登场

循环外先读第一个数，直接当最大值。

### ② 后来者挨个挑战

更大就换擂主： max = x;

### ③ 循环从 2 开始

第 1 个数已是擂主，只需再读 n−1 个。

# 求平均值

sum 放循环外 · double 防整数除法

```
// 读 n 个成绩，求平均分
int n, i;
double score, sum, avg;
scanf("%d", &n);
sum = 0;
// ↑ sum 放在循环外面
for (i = 1; i <= n; i++) {
    scanf("%lf", &score);
    sum = sum + score;
}
avg = sum / n;
printf("平均 = %.2f", avg);
```

## ① sum 放循环外

sum = 0; 必须在 for 之前。

放循环里会被反复清零。

## ② sum 声明成 double

int 相加再除，小数直接被扔掉。

改成 double 才能留住小数。

## ③ 除法要有一端是小数

sum / n ： sum 是 double ，结果带小数。

3 / 2 得 1 ， 3.0 / 2 才得 1.5 。

# 打印乘法口诀表



**任务** 输出九九乘法口诀表（三角形，第 i 行只打到 i×i ）

```
int i, j;
for (i = 1; i <= 9; i++) {
    // 外层：一共 9 行
    for (j = 1; j <= i; j++) {
        // 内层：第 i 行打 i 个
        printf("%d*%d=%d ", j, i, i*j);
    }
    printf("%c", 10);
    // 打完一行换个行
}
```

## 运行结果（前 5 行）

```
1*1=1
1*2=2 2*2=4
1*3=3 2*3=6 3*3=9
1*4=4 2*4=8 3*4=12 4*4=16
1*5=5 2*5=10 3*5=15 4*5=20 5*5=25
```

…… 后面 4 行同理

## 为什么是三角形

内层写成 j <= i ，不是 j <= 9 。

口诀表只留左下角一半，不重复。

总次数 1+2+…+9 = 45 次。

# 水仙花数



**任务** 找出所有水仙花数：三位数 = 它各位数字的立方和（如 153 = 1+125+27 ）

```
int i, a, b, c;
for (i = 100; i <= 999; i++) {
    a = i / 100;
    // 百位
    b = i / 10 % 10;
    // 十位
    c = i % 10;
    // 个位
    if (i == a*a*a + b*b*b + c*c*c) {
        printf("%d ", i);
    }
}
```

## 运行结果

```
153 370 371 407
```

## 拆位的三板斧

百位：i / 100

十位：i / 10 % 10

个位：i % 10

## 思路

循环把 900 个三位数挨个看一遍。

if 只挑出符合条件的那 4 个。

# 鸡兔同笼

双重循环穷举

哈尔滨工程大学

**任务** 笼子里共 35 个头、 94 只脚，问鸡和兔各有几只？

```
int x, y;
for (x = 0; x <= 35; x++) {
    // 假设鸡有 x 只
    for (y = 0; y <= 35; y++) {
        // 假设兔有 y 只
        if (x + y == 35) {
            if (2*x + 4*y == 94) {
                printf("鸡%d 兔%d", x, y);
            }
        }
    }
}
```

## 运行结果

鸡 23 兔 12

## 穷举的思路

把鸡的只数从 0 试到 35 。

每一种鸡再配一遍兔的只数。

两层一共试 36 × 36 = 1296 种。

## if 才是裁判

头数和脚数都对上才打印。

循环负责「试」，if 负责「判」。

# 例 1 ：1~100 求和

累加器第一次完整登场



**例 1** 计算 1 + 2 + 3 + …… + 100 的值

```
int i, sum;
sum = 0;
// ① 累加器先置 0
for (i = 1; i <= 100; i++) {
    // ② 一共跑 100 圈
    sum += i;
    // ③ 每圈加一个 i
}
printf("和 = %d", sum);
// 输出 和 = 5050
// 验算：(1+100) × 100 / 2 = 5050
```

## 运行结果

**和 = 5050**

## 三步不能少

① sum = 0 写在 for 之前

② 循环条件 i <= 100

③ sum += i 写在循环体里

## 想一想

只加偶数，循环怎么改？

起点改 2、步长改 2 即可。

# 例 2 ：n 的阶乘

累乘器 · double 防溢出

哈尔滨工程大学

**例 2** 从键盘读入 n ，输出 n! = n × (n−1) × …… × 1 的值

```
int n, i;
double fact;
scanf("%d", &n);
fact = 1;
// ① 累乘器先置 1，不是 0
for (i = 1; i <= n; i++) {
    fact = fact * i;
    // ② 每圈乘一个 i
}
printf("%d! = %.0f", n, fact);
// 输入 10 输出 10! = 3628800
```

## 运行结果

输入 10 → 10! = 3628800

输入 5 → 5! = 120

## 两个易错点

fact 置成 0 ：结果永远是 0 。

用 int 装： 13! 就溢出变负数。

所以 fact 声明成 double 。

## 输出格式

%.0f 表示小数位为 0 位。

double 必须用 %f ，不能用 %d 。

# 例 3 ：n 个成绩平均值



**例 3** 先输入学生人数 n ，再输入 n 个成绩，输出平均分（保留 2 位小数）

```
int n, i;
double score, sum, avg;
scanf("%d", &n);
sum = 0;
// ① sum 放在循环外面
for (i = 1; i <= n; i++) {
    scanf("%lf", &score);
    // double 用 %lf
    sum = sum + score;
}
avg = sum / n;
printf("平均 = %.2f", avg);
```

## 运行结果

```
n = 3   成绩 90 80 70
平均 = 80.00
n = 3   成绩 90 81 70
平均 = 80.33
```

## 两个陷阱

sum 写成 int ：小数全被丢掉。

%.2f 才是保留两位小数。

printf 里 double 用 %f 输出。

# 例 4 ：最值擂台



**例 4** 输入 n 个整数，输出其中的最大值和最小值

```
int n, i, x, max, min;
scanf("%d", &n);
scanf("%d", &x);
max = x;
min = x;
//  第一个数同时当两个擂主
for (i = 2; i <= n; i++) {
    scanf("%d", &x);
    if (x > max) { max = x; }
    if (x < min) { min = x; }
}
printf("最大 %d 最小 %d", max, min);
```

## 运行结果

n = 5　数据 3 9 1 7 5

最大 9 最小 1

## 为什么要两个 if

一个数是大是小要各判一次。

写成 else if 就会漏判一半情况。

## 初始化不靠猜

拿第一个真实数据当擂主。

比自己拍一个 max = 0 稳妥得多。

# 例 5 ：调和级数

1.0 / i 不能写成 1 / i

哈爾濱工程大學

**例 5** 输入 n ，求 1 + 1/2 + 1/3 + ..... + 1/n 的值（保留 6 位小数）

```
int n, i;
double sum;
scanf("%d", &n);
sum = 0;
for (i = 1; i <= n; i++) {
    sum = sum + 1.0 / i;
    // 必须写 1.0，不能写 1
}
printf("H = %.6f", sum);
// n = 10 时输出 H = 2.928968
// 写成 1 / i 会怎样？看右边
```

**正确： sum + 1.0 / i**

n = 10 → H = 2.928968

**错误： sum + 1 / i**

n = 10 → H = 1.000000

1 和 i 都是整数，1 / i 按整数除法算。

i = 1 时得 1 ，i > 1 时全得 0。

最后只剩第一项的 1。

**记住**

只要除号有一端是小数，结果就是小数。

# 常见错误盘点

这 6 条占初学者错误的九成

哈尔滨工程大学

✗ for (i = 0; i < n; i++); 末尾多一个分号 → 循环体变成空语句

✗ for (i = 1; i >= 0; i++) 条件永远成立 → 程序停不下来

✗ 循环体里又写 i = i + 2 → 步进改了两次，次数全乱

✗ sum 忘了写 sum = 0 → 初值是内存里的随机垃圾

✗ sum 和 n 都是 int 时 sum / n → 小数被直接丢掉

✗ i < n 与 i <= n 差一次 → 不是多跑一轮就是少跑一轮

# 小结

for 三要素：初始值、条件、步进；初始化只做一次。

累加器先置 0 、累乘器先置 1 ，都写在循环外面。

嵌套 for ：外层走一步，内层走一圈，总次数相乘。

下一课预告

**事先不知道要循环几次，怎么办？**

for 适合「次数已知」的场景，由计数变量决定跑几圈。

还有一种循环只靠条件决定何时停下，下一课专门讲它。

哈爾濱工程大學